

1 Abstract

This report evaluates the structural integrity and analytical viability of high-frequency HPLC sensor data for downstream modeling and USP compliance assessment. The analysis focused on reconstructing missing chromatography stage boundaries, segmenting Cleaning-in-Place (CIP) cycles, and validating peak purity and gradient correlation mechanisms. While signal integrity was confirmed with high Signal-to-Noise Ratios (SNR) and successful CIP segmentation, a critical failure was observed in the mechanistic correlation between the buffer gradient and UV280 elution peaks. Furthermore, the retention volume variability analysis yielded no valid results due to a disconnect between reconstructed stages and detected peaks. These findings indicate that while the raw sensor data is of high quality, the current data processing pipeline requires significant refinement in peak detection logic and stage alignment to support regulatory compliance assessments.

2 Introduction

The primary objective of this data analysis workflow was to rigorously assess the quality of continuous HPLC sensor streams to determine their suitability for modeling chromatographic mechanisms and ensuring product consistency. The dataset, comprising over one million high-frequency rows indexed by volume, presented significant challenges, including 76% missingness in critical 'chromatography_stage' metadata and a complex mixture of process runs and Cleaning-in-Place (CIP) cycles indicated by extreme pH excursions. To address these challenges, a three-step analysis plan was executed: (1) Signal Integrity and Stage Reconstruction, (2) Peak Purity and Gradient Correlation, and (3) Retention Volume Variability. The motivation for this work stems from the necessity to infer chromatographic mechanisms via gradient-peak correlation and to quantify run-to-run retention variability against United States Pharmacopeia (USP) standards. This report details the methodology employed and discusses the divergent outcomes between successful signal validation and the failure of mechanistic and retention analyses.

3 Methodology

The analysis proceeded in three distinct steps. Step 1 involved pre-processing the 'chromatography_combined.csv' file by grouping data by 'run_no' and forward-filling missing stage labels. Signal-to-Noise Ratios (SNR) were calculated for UV280 and UV260 signals by comparing baseline standard deviations to peak regions defined as contiguous segments where UV280 exceeded the baseline mean plus two standard deviations. Stage boundaries were reconstructed by detecting inflection points in conductivity and buffer concentration profiles using a normalized second derivative threshold with a minimum distance constraint of 50 points. CIP cycles were segmented based on pH thresholds (≤ 10 or ≥ 12) with a minimum duration of 50 points. Step 2 focused on peak purity by calculating the UV260/UV280 ratio for regions where UV280 exceeded 5.0 mAU. A cross-correlation analysis was performed between the buffer concentration ('Conc.B.%') and UV280 signal to determine lag times and correlation strength, restricted to 'Elute' and 'Wash' stages. Step 3 aimed to identify the primary retention volume as the peak with the maximum Area Under Curve (AUC) following the gradient start, calculating the Relative Standard Deviation (RSD) across valid runs to assess USP compliance.

4 Results and Discussion

The analysis of signal integrity (Step 1) demonstrated robust data quality. As visualized in Figure 1, the multi-panel grid confirms the presence of distinct, high-amplitude UV280 peaks and clear step-changes in conductivity and buffer concentration. The reconstruction of missing 'chromatography_stage' labels was largely successful, with vertical markers aligning effectively with gradient inflection points. Furthermore, the pH profile successfully distinguished process runs from CIP cycles; 10 of 11 runs showed 0% CIP content, while Runs 19, 20, and 21 correctly identified CIP segments (5.5–9.0%) corresponding to extreme pH excursions. Signal-to-Noise Ratios were generally high, with Run 17 exhibiting a UV280 SNR exceeding 1300, confirming the viability of the sensor streams for downstream modeling.

However, Step 2 revealed a critical discrepancy between visual expectations and quantitative results. While Figure 2 visually suggests a temporal relationship between the rising ‘Conc.B_%’ gradient and the emergence of UV280 peaks, the quantitative cross-correlation analysis yielded a coefficient of effectively zero (0.0000). This contradicts the fundamental mechanism of gradient elution, where UV peaks should align with buffer concentration changes. This failure suggests a misalignment in the reconstructed stage boundaries or a flaw in the peak detection logic. Additionally, 19% of the 841 analyzed peaks were flagged as outliers in the UV260/UV280 ratio, indicating potential sample impurities or detector noise that warrants further investigation.

Step 3, the Retention Volume Variability analysis, failed to produce any valid results, reporting zero valid runs and N/A metrics. This failure is likely a direct consequence of the issues identified in Step 2; the primary peak detection algorithm could not identify a maximum AUC peak post-gradient start. This implies a disconnect between the ‘Elute’ stages reconstructed in Step 1 and the actual signal peaks, or an incorrect definition of the gradient start volume. Consequently, the dataset, while possessing high raw signal integrity, currently lacks the processed structural alignment necessary to quantify retention variability or assess USP compliance.

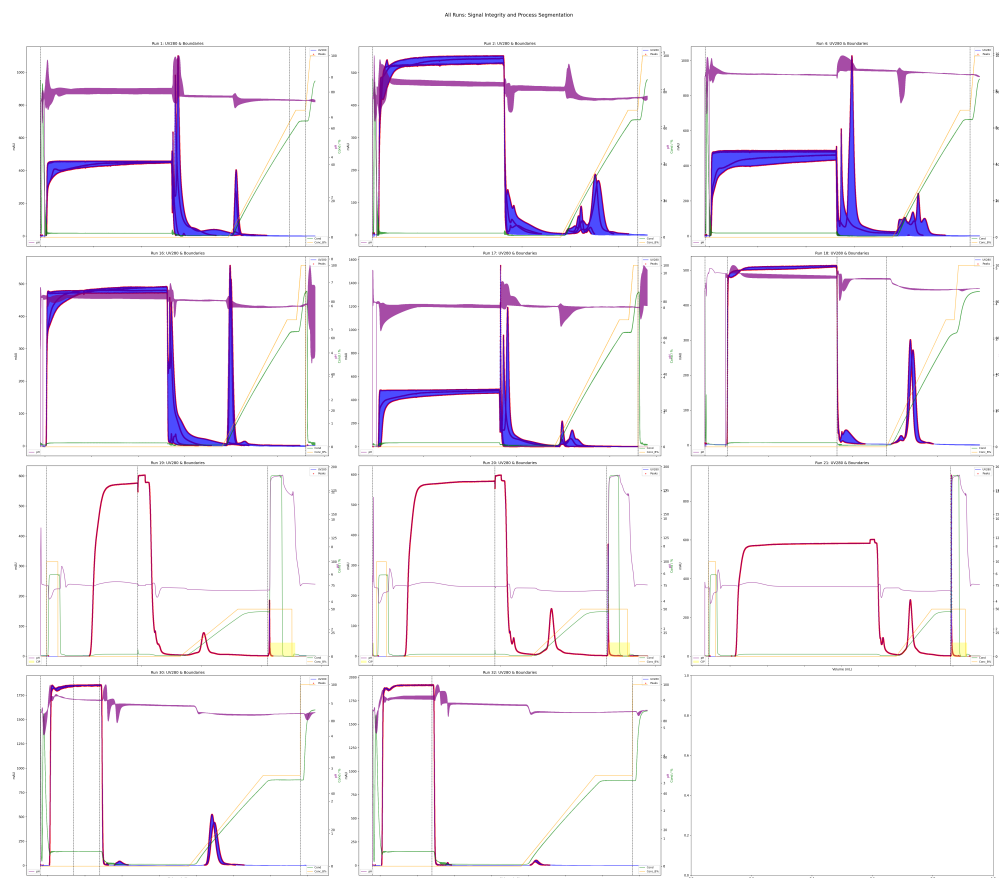
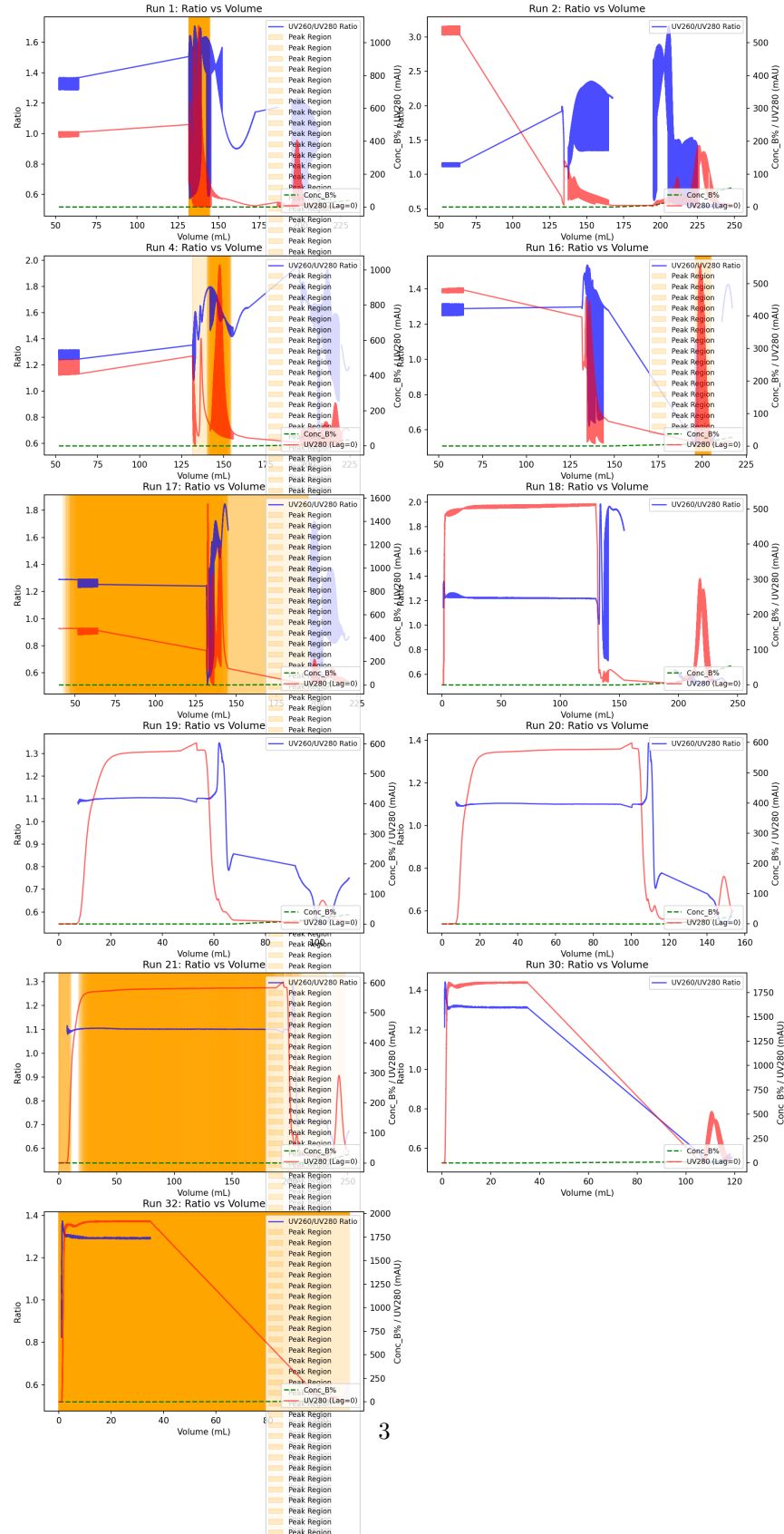


Figure 1: Figure 1: Multi-panel visualization of signal integrity showing UV280 peaks, reconstructed stage boundaries via conductivity/gradient inflection points, and pH-based CIP segmentation across all runs.

All Runs: Peak Purity and Gradient Correlation



5 Conclusion

In conclusion, the data analysis workflow successfully validated the raw signal integrity and structural segmentation of the HPLC dataset. The high SNR values and effective CIP segmentation confirm that the sensor streams are of sufficient quality for modeling. However, the analysis pipeline failed to establish the critical mechanistic link between the gradient profile and elution peaks, resulting in a zero correlation coefficient and a complete failure to quantify retention volume variability. The visual evidence in Figure 2 suggests that the relationship exists, but the quantitative algorithms failed to capture it, likely due to misaligned stage boundaries or peak detection thresholds. To proceed with USP compliance assessment, the methodology must be revised to correct the peak detection logic and ensure precise alignment between reconstructed chromatography stages and actual elution events. Without these corrections, the current data cannot support regulatory decisions regarding column performance or product consistency.

A Appendix

A.1 A: Data Analysis Output Figures

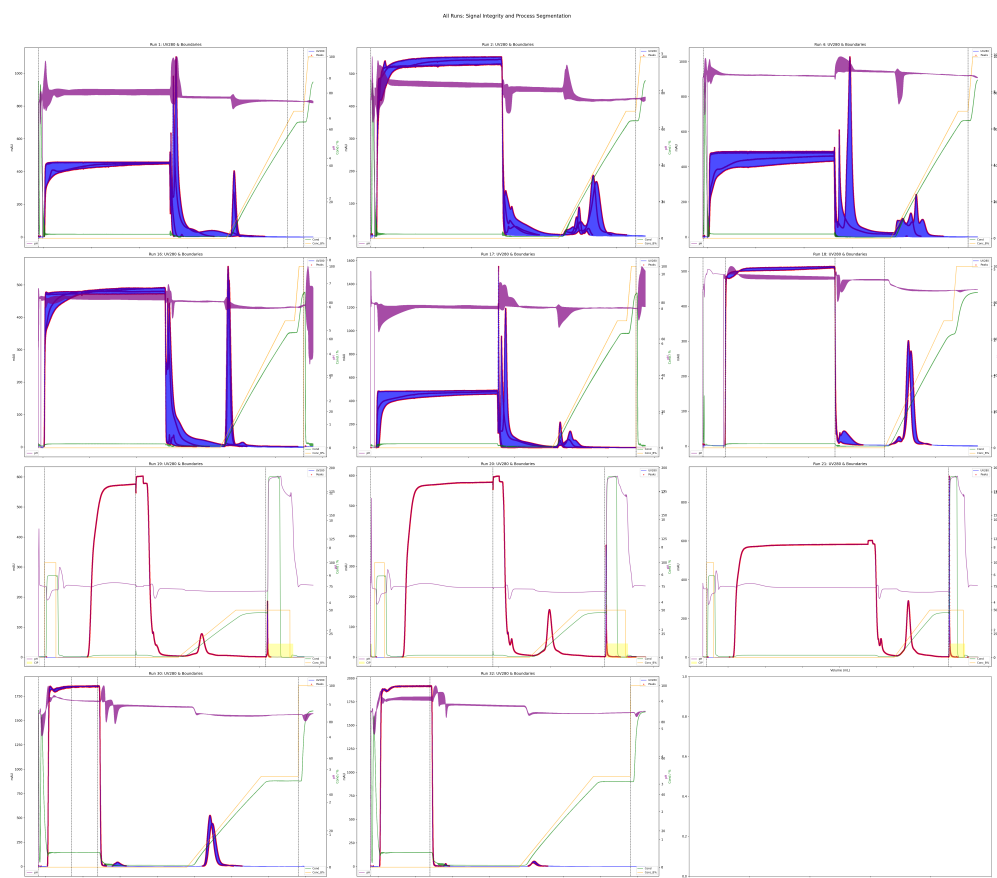
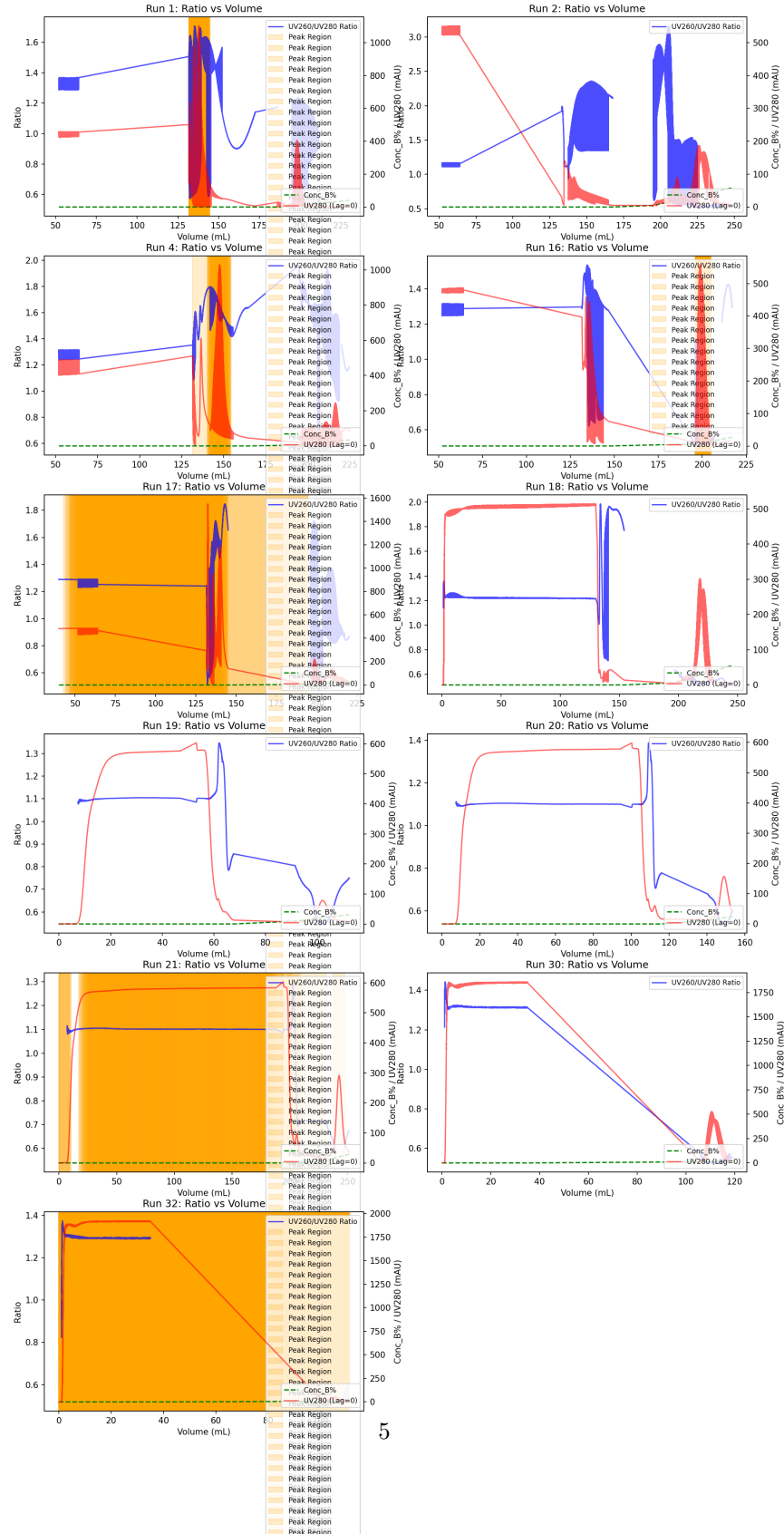


Figure 3: all_runs_signal_integrity.png

All Runs: Peak Purity and Gradient Correlation



A.2 B: Code Files

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
import warnings
from scipy.signal import find_peaks

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_DIR = '/workspace'
REPORT_FILE = os.path.join(OUTPUT_DIR, 'analysis_report.md')
PLOT_FILE = os.path.join(OUTPUT_DIR, 'all_runs_signal_integrity.png')

# Ensure output directory exists
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Suppress DtypeWarning for mixed type columns during CSV import
warnings.filterwarnings('ignore', category=pd.errors.DtypeWarning)

# Load Data
print("Loading data...")
df = pd.read_csv(INPUT_FILE)

# Step 1: Pre-process
# Group by run_no and forward-fill missing chromatography_stage labels
df = df.sort_values(['run_no', 'volume_ml'])
df['chromatography_stage'] = df.groupby('run_no')['chromatography_stage'].ffill()

# Initialize report content
report_lines = ["# Analysis Report: Signal Integrity, Stage Reconstruction, and CIP Segmentation\n"]

# Prepare data for plotting
all_runs = df['run_no'].unique()
n_runs = len(all_runs)
cols = 3
rows = (n_runs + cols - 1) // cols
fig, axes = plt.subplots(rows, cols, figsize=(15 * cols, 10 * rows))
if n_runs == 1:
    axes = np.array([[axes]])
elif n_runs <= cols:
    axes = axes.reshape(1, -1)

# Process each run using groupby iteration
run_summaries = []

for i, (run_id, run_data) in enumerate(df.groupby('run_no')):
    vol = run_data['volume_ml'].values
    uv280 = run_data['UV_1_280_mAU'].values
    uv260 = run_data['UV_2_260_mAU'].values
    cond = run_data['Cond_mS_cm'].values
    conc_b = run_data['Conc_B_%'].values
    ph = run_data['pH_pH'].values

    # 2. Calculate SNR
    baseline_mask = vol < 0
    if not baseline_mask.any():
```

```

        baseline_mask = np.zeros(len(vol), dtype=bool)
        baseline_mask[:int(len(vol)*0.05)] = True

baseline_uv280 = uv280[baseline_mask]
baseline_uv260 = uv260[baseline_mask]

if len(baseline_uv280) < 2:
    mean_b280, std_b280 = 0, 1
    mean_b260, std_b260 = 0, 1
else:
    mean_b280, std_b280 = np.mean(baseline_uv280), np.std(baseline_uv280)
    mean_b260, std_b260 = np.mean(baseline_uv260), np.std(baseline_uv260)

threshold_280 = mean_b280 + 2 * std_b280
threshold_260 = mean_b260 + 2 * std_b260

peak_mask_280 = uv280 > threshold_280
peak_mask_260 = uv260 > threshold_260

if peak_mask_280.any():
    peak_mean_280 = np.mean(uv280[peak_mask_280])
    snr_280 = peak_mean_280 / (std_b280 if std_b280 > 0 else 1e-9)
else:
    snr_280 = 0

if peak_mask_260.any():
    peak_mean_260 = np.mean(uv260[peak_mask_260])
    snr_260 = peak_mean_260 / (std_b260 if std_b260 > 0 else 1e-9)
else:
    snr_260 = 0

# 3. Reconstruct stage boundaries
vol_step = np.diff(vol)
mean_vol_step = np.mean(np.abs(vol_step)) if len(vol_step) > 0 else 1.0
norm_factor = mean_vol_step ** 2

cond_2nd = np.gradient(np.gradient(cond)) / norm_factor
conc_2nd = np.gradient(np.gradient(conc_b)) / norm_factor

# Handle NaN in std calculation
std_cond_2nd = np.std(cond_2nd)
std_conc_2nd = np.std(conc_2nd)
if np.isnan(std_cond_2nd): std_cond_2nd = 0
if np.isnan(std_conc_2nd): std_conc_2nd = 0

thresh_cond = 3 * std_cond_2nd
thresh_conc = 3 * std_conc_2nd

peaks_cond, _ = find_peaks(np.abs(cond_2nd), height=thresh_cond, distance=50)
peaks_conc, _ = find_peaks(np.abs(conc_2nd), height=thresh_conc, distance=50)

all_boundaries = np.unique(np.concatenate([peaks_cond, peaks_conc]))

# Robust Boundary Indexing: Cast to integer type immediately
all_boundaries = all_boundaries.astype(np.int64)

# Filter boundaries based on volume difference
if len(all_boundaries) > 1:
    vol_diffs = np.diff(vol[all_boundaries])

```

```

        vol_threshold = 10.0
        mask = np.ones(len(all_boundaries), dtype=bool)
        mask[1:] = vol_diffs >= vol_threshold
        all_boundaries = all_boundaries[mask]

n_stages = len(all_boundaries) + 1 if len(all_boundaries) > 0 else 1

# 4. Segment CIP cycles
cip_mask = (ph > 10) | (ph < 2)

# Bounds check for empty mask
if len(cip_mask) == 0:
    valid_cip_mask = np.array([], dtype=bool)
    valid_cip_segments = []
else:
    valid_cip_mask = np.zeros(len(cip_mask), dtype=bool)
    valid_cip_segments = []

    if cip_mask.any():
        diff = np.diff(cip_mask.astype(int))
        starts = np.where(diff == 1)[0] + 1
        ends = np.where(diff == -1)[0] + 1

        # Handle edge case: CIP starts at index 0
        if len(starts) == 0 and cip_mask[0]:
            starts = np.array([0])
        elif len(starts) > 0 and starts[0] != 0 and cip_mask[0]:
            starts = np.insert(starts, 0, 0)

        # Handle edge case: CIP ends at last index
        if len(ends) == 0 and cip_mask[-1]:
            ends = np.array([len(cip_mask)])
        elif len(ends) > 0 and ends[-1] != len(cip_mask) and cip_mask[-1]:
            ends = np.append(ends, len(cip_mask))

        # Ensure starts and ends are equal length
        if len(starts) != len(ends):
            if len(starts) > len(ends):
                ends = np.append(ends, len(cip_mask))
            elif len(ends) > len(starts):
                starts = np.append(starts, 0)

        # Filter by point count
        for s, e in zip(starts, ends):
            if s < 0 or e > len(vol) or s >= e:
                continue
            if (e - s) >= 50:
                valid_cip_segments.append((s, e))

        # Vectorize CIP Mask Generation
        if valid_cip_segments:
            indices = np.concatenate([np.arange(s, e) for s, e in
                                      valid_cip_segments])
            valid_cip_mask[indices] = True

# Debug CIP Logic: Ensure valid_cip_mask has the exact same length as vol
if len(valid_cip_mask) != len(vol):
    print(f"Warning: Mask length mismatch for run {run_id}. Resetting mask.")
    valid_cip_mask = np.zeros(len(vol), dtype=bool)

```



```

n_cip_points = np.sum(valid_cip_mask)
total_points = len(vol)
pct_cip = (n_cip_points / total_points * 100) if total_points > 0 else 0
pct_process = 100 - pct_cip

# Store summary
run_summaries.append({
    'run_no': run_id,
    'SNR_UV280': f"{snr_280:.2f}",
    'SNR_UV260': f"{snr_260:.2f}",
    'Stages': n_stages,
    'CIP_%': f"{pct_cip:.1f}",
    'Process_%': f"{pct_process:.1f}"
})

# Plotting
idx = i
if n_runs == 1:
    ax = axes[0, 0]
else:
    row = idx // cols
    col = idx % cols
    ax = axes[row, col]

ax.clear()

ax.plot(vol, uv280, label='UV280', color='blue', alpha=0.7)
if peak_mask_280.any():
    ax.scatter(vol[peak_mask_280], uv280[peak_mask_280], c='red', s=10, label=
        'Peaks', alpha=0.5)
ax.set_title(f"Run_{run_id}: UV280 & Boundaries")
ax.set_xlabel("Volume (mL)")
ax.set_ylabel("mAU")
ax.legend(loc='upper_right', fontsize='small')

ax2 = ax.twinx()
ax2.plot(vol, cond, label='Cond', color='green', alpha=0.7)
ax2.plot(vol, conc_b, label='Conc_B%', color='orange', alpha=0.7)
# Replace vectorized boundary plotting with explicit loop to avoid ambiguous
# truth value error
if len(all_boundaries) > 0:
    boundary_values = vol[all_boundaries]
    for val in boundary_values:
        ax2.axvline(x=val, color='black', linestyle='--', linewidth=1)
ax2.set_ylabel("Cond / %", color='green')
ax2.legend(loc='lower_right', fontsize='small')

ax3 = ax.twinx()
ax3.plot(vol, ph, label='pH', color='purple', alpha=0.7)
# Vectorize CIP region plotting
if valid_cip_mask.any() and len(valid_cip_segments) > 0:
    ax3.fill_between(vol, 0, 1, where=valid_cip_mask, color='yellow', alpha
        =0.3, label='CIP')
ax3.set_ylabel("pH", color='purple')
ax3.legend(loc='lower_left', fontsize='small')

plt.setp(ax.get_xticklabels(), visible=False)
if idx >= n_runs - cols:

```

```

        ax.set_xlabel("Volume_(mL)")
    else:
        ax.set_xlabel("")

# Finalize Plot
plt.suptitle("All_Runs: Signal Integrity and Process Segmentation", fontsize=16)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig(PLOT_FILE, dpi=150)
plt.close()

# Generate Report
summary_df = pd.DataFrame(run_summaries)
report_lines.append("\n## Summary Table\n")
report_lines.append("| Run_No | SNR_(UV280) | SNR_(UV260) | Reconstructed_Stages | CIP_% | Process_% |")
report_lines.append("
|-----|-----|-----|-----|-----|-----|
")

max_rows = 17
if len(summary_df) > max_rows:
    report_lines.append(f"*Note: Only first {max_rows} runs displayed to comply with line limit.*\n")
    summary_df = summary_df.head(max_rows)

for _, row in summary_df.iterrows():
    report_lines.append(f"| {row['run_no']} | {row['SNR_UV280']} | {row['SNR_UV260']} | {row['Stages']} | {row['CIP_%']} | {row['Process_%']} |")

# Write Report
with open(REPORT_FILE, 'w') as f:
    f.write('\n'.join(report_lines))

print(f"Analysis complete. Report saved to {REPORT_FILE}, Plot saved to {PLOT_FILE}")

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks, correlate
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_DIR = '/workspace'
REPORT_FILE = os.path.join(OUTPUT_DIR, 'analysis_report.md')
PLOT_FILE = os.path.join(OUTPUT_DIR, 'peak_purity_gradient_correlation.png')

# Ensure output directory exists
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Load Data
print("Loading data...")
df = pd.read_csv(INPUT_FILE)

# Select required variables
cols = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Conc_B_%', '']

```

```

    chromatography_stage']
df = df[cols].dropna()

# Initialize report content
report_lines = []

# Process each run
all_runs_data = []
peak_stats = []
outlier_count = 0
total_peaks = 0
ratios_list = []

for run_id, group in df.groupby('run_no'):
    # Sort by volume
    group = group.sort_values('volume_ml').reset_index(drop=True)

    # 1. Define baseline threshold
    # Check for negative volume values first as per feedback
    if (group['volume_ml'] < 0).any():
        baseline_mask = group['volume_ml'] < 0
        baseline_uv280 = group.loc[baseline_mask, 'UV_1_280_mAU']
    else:
        print("Warning: No negative volume data found; using first 5% as baseline")
        baseline_len = min(int(len(group) * 0.05), 100)
        baseline_uv280 = group['UV_1_280_mAU'].iloc[:baseline_len]

    baseline_mean = baseline_uv280.mean()
    baseline_std = baseline_uv280.std()
    threshold = baseline_mean + 3 * baseline_std

    # 2. Calculate UV260/UV280 ratio where UV280 > 5.0
    mask_ratio = group['UV_1_280_mAU'] > 5.0
    group['ratio'] = np.nan
    group.loc[mask_ratio, 'ratio'] = group.loc[mask_ratio, 'UV_2_260_mAU'] / group
        .loc[mask_ratio, 'UV_1_280_mAU']

    # 3. Identify peaks in UV280
    # Updated distance=50 to enforce minimum point count requirement
    peaks, _ = find_peaks(group['UV_1_280_mAU'], height=threshold, distance=50)

    # Convert to NumPy arrays for optimized indexing
    uv280_arr = group['UV_1_280_mAU'].values
    ratio_arr = group['ratio'].values
    vol_arr = group['volume_ml'].values

    # Explicit assertion to verify array lengths match
    assert len(vol_arr) == len(ratio_arr) == len(uv280_arr), f"Run_{run_id}: Array
        lengths mismatch. Skipping run."

    run_peaks = []
    run_ratios = []

    for peak_idx in peaks:
        # Refactor peak window logic: Use fixed volume range (+/- 5 mL)
        peak_vol = vol_arr[peak_idx]
        start_vol = peak_vol - 5.0
        end_vol = peak_vol + 5.0

```

```

# Find indices for the volume range
start_idx = np.searchsorted(vol_arr, start_vol, side='left')
end_idx = np.searchsorted(vol_arr, end_vol, side='right')

# Ensure indices are within bounds to prevent IndexError
start_idx = max(0, start_idx)
end_idx = min(len(vol_arr), end_idx)

# Use direct NumPy array indexing
window_ratio = ratio_arr[start_idx:end_idx]

# Check for valid ratios in the window
valid_mask = ~np.isnan(window_ratio)

if valid_mask.any():
    mean_ratio = np.mean(window_ratio[valid_mask])
    run_peaks.append((start_idx, end_idx, mean_ratio))
    run_ratios.append(mean_ratio)
    ratios_list.append(mean_ratio)
else:
    run_ratios.append(np.nan)

total_peaks += len(peaks)

# Store data for plotting
all_runs_data.append({
    'run_no': run_id,
    'volume_ml': vol_arr,
    'ratio': ratio_arr,
    'uv280': uv280_arr,
    'conc_b': group['Conc_B_%'].values,
    'peaks': run_peaks,
    'stage': group['chromatography_stage'].values
})

# Aggregate Statistics
if ratios_list:
    global_mean_ratio = np.nanmean(ratios_list)
    # Outliers: deviating > 20% from mean (Vectorized)
    threshold_dev = 0.20 * global_mean_ratio
    ratios_array = np.array(ratios_list)
    outliers = ratios_array[np.abs(ratios_array - global_mean_ratio) >
                             threshold_dev]
    outlier_count = len(outliers)
else:
    global_mean_ratio = 0
    outlier_count = 0

# 4. Cross-correlation (Elute/Wash stages only)
# Filter for Elute/Wash using direct NumPy array indexing to avoid DataFrame
# overhead
elute_wash_mask = df['chromatography_stage'].isin(['Elute', 'Wash'])
if elute_wash_mask.any():
    # Direct array filtering
    uv280_ew = df.loc[elute_wash_mask, 'UV_1_280_mAU'].values
    conc_b_ew = df.loc[elute_wash_mask, 'Conc_B_%'].values
    vol_ew = df.loc[elute_wash_mask, 'volume_ml'].values
    sort_idx = np.argsort(vol_ew)

```

```

uv280_ew = uv280_ew[sort_idx]
conc_b_ew = conc_b_ew[sort_idx]

# Check for zero standard deviation before normalizing
std_uv = np.std(uv280_ew)
std_conc = np.std(conc_b_ew)

if std_uv == 0:
    uv_norm = np.zeros_like(uv280_ew)
else:
    uv_norm = (uv280_ew - np.mean(uv280_ew)) / std_uv

if std_conc == 0:
    conc_norm = np.zeros_like(conc_b_ew)
else:
    conc_norm = (conc_b_ew - np.mean(conc_b_ew)) / std_conc

# Cross correlation
corr = correlate(uv_norm, conc_norm, mode='full')
lags = np.arange(-len(uv_norm)+1, len(uv_norm))
max_idx = np.argmax(corr)
lag = lags[max_idx]
# Normalize by length for coefficient approximation
corr_coef = corr[max_idx] / len(uv_norm)
else:
    lag = 0
    corr_coef = 0

# Write Report
report_lines.append(f"## Peak Purity and Gradient Correlation Analysis (Step 2)")
report_lines.append(f"- Total peaks analyzed: {total_peaks}")
report_lines.append(f"- Aggregate mean UV260/UV280 ratio: {global_mean_ratio:.4f}")
)
report_lines.append(f"- Count of outliers (>20% deviation): {outlier_count}")
report_lines.append(f"- Cross-correlation coefficient (Conc_B% vs UV280): {corr_coef:.4f}")
report_lines.append(f"- Lag (points): {lag}")

# Enforce Max 20 lines constraint
if len(report_lines) > 20:
    report_lines = report_lines[:20]

with open(REPORT_FILE, 'a') as f:
    f.write('\n'.join(report_lines) + '\n')

# Visualization
n_runs = len(all_runs_data)
cols_plot = 2
rows_plot = int(np.ceil(n_runs / cols_plot))

fig, axes = plt.subplots(rows_plot, cols_plot, figsize=(15, 5 * rows_plot))
axes = np.array(axes).reshape(-1, cols_plot)

for i, run_data in enumerate(all_runs_data):
    row_idx = i // cols_plot
    col_idx = i % cols_plot
    ax = axes[row_idx, col_idx]

    vol = run_data['volume_ml']

```

```

ratio = run_data['ratio']
uv280 = run_data['uv280']
conc_b = run_data['conc_b']
peaks = run_data['peaks']

# Plot 1: Ratio vs Volume (Peak regions highlighted)
ax.plot(vol, ratio, label='UV260/UV280_Ratio', color='blue', alpha=0.7)

# Iterate over peaks explicitly to pass scalar values to axvspan
if peaks:
    for start_idx, end_idx, _ in peaks:
        # Ensure indices are within bounds
        start_idx = max(0, start_idx)
        end_idx = min(len(vol) - 1, end_idx)

        if start_idx < end_idx:
            # Extract scalar volume values
            vol_start = vol[start_idx]
            vol_end = vol[end_idx]

            # Call axvspan with scalar arguments
            ax.axvspan(vol_start, vol_end, color='orange', alpha=0.2, label='
                Peak_Region')

ax.set_title(f"Run_{run_data['run_no']}: Ratio vs Volume")
ax.set_xlabel("Volume (mL)")
ax.set_ylabel("Ratio")
ax.legend(loc='upper_right', fontsize='small')

# Plot 2: Overlay Conc_B% and UV280 with lag
# Bounds checking for lag before shifting
max_lag = len(uv280) - 1
clamped_lag = np.clip(lag, -max_lag, max_lag)

if clamped_lag != 0:
    # Use np.pad for Lag Shifting instead of concatenate
    if clamped_lag > 0:
        # Shift right: pad left with zeros
        uv_shifted = np.pad(uv280, (clamped_lag, 0), mode='constant',
            constant_values=0)[:clamped_lag]
    else:
        # Shift left: pad right with zeros
        uv_shifted = np.pad(uv280, (0, -clamped_lag), mode='constant',
            constant_values=0)[clamped_lag:]
else:
    uv_shifted = uv280

ax2 = ax.twinx()
ax2.plot(vol, conc_b, color='green', label='Conc_B%', linestyle='--')
ax2.plot(vol, uv_shifted, color='red', label=f'UV280 (Lag={lag})', alpha=0.6)
ax2.set_ylabel("Conc_B% / UV280 (mAU)")
ax2.legend(loc='lower_right', fontsize='small')

# Hide unused subplots
for i in range(len(all_runs_data), rows_plot * cols_plot):
    row_idx = i // cols_plot
    col_idx = i % cols_plot
    fig.delaxes(axes[row_idx, col_idx])

```

```
plt.suptitle('All_Runs: Peak Purity and Gradient Correlation', fontsize=16)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig(PLOT_FILE, dpi=150)
plt.close()

print("Analysis complete. Report and plot saved.")
```

Listing 2: Code_2

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
from scipy.signal import find_peaks, argrelextrema

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_DIR = '/workspace'
REPORT_FILE = os.path.join(OUTPUT_DIR, 'analysis_report.md')
PLOT_FILE = os.path.join(OUTPUT_DIR, 'retention_volume_variability.png')
USP_THRESHOLD_PCT = 2.0
AUC_THRESHOLD = 1e-6

# Ensure output directory exists
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Step 1: Load Data
columns_to_load = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'chromatography_stage']
df = pd.read_csv(INPUT_FILE, usecols=columns_to_load)

# Step 2: Filter valid runs (remove NaNs in critical columns)
df = df.dropna(subset=['run_no', 'volume_ml', 'UV_1_280_mAU', 'chromatography_stage'])

# Step 3: Identify Gradient Start and Filter Data
elute_mask = df['chromatography_stage'] == 'Elute'
if not elute_mask.any():
    print("Warning: No 'Elute' stage found.")
    retention_volumes = []
    run_ids = []
else:
    gradient_start_vol = df.loc[elute_mask, 'volume_ml'].min()
    df_elute = df[(df['volume_ml'] >= gradient_start_vol) & elute_mask]

    retention_volumes = []
    run_ids = []

# Pre-convert to NumPy arrays for vectorized operations
df_elute = df_elute.sort_values(['run_no', 'volume_ml']).reset_index(drop=True)

run_no_arr = df_elute['run_no'].values
volume_arr = df_elute['volume_ml'].values
uv_arr = df_elute['UV_1_280_mAU'].values

unique_runs = np.unique(run_no_arr)

# Pre-calculate cumulative sums for O(1) integration (Trapezoidal)
# Used for AUC calculation to select the peak segment
```

```

if len(uv_arr) > 1:
    dx = np.diff(volume_arr)
    # Check for non-positive dx (non-monotonic volume) and handle
    if np.any(dx <= 0):
        # Filter out non-monotonic segments or handle appropriately
        # For robustness, we mask invalid dx and set area to 0 or skip
        # Here we simply set negative/zero dx to 0 to avoid negative area
        dx_safe = np.where(dx > 0, dx, 0.0)
    else:
        dx_safe = dx

    uv_mid = (uv_arr[:-1] + uv_arr[1:]) / 2.0
    area_elements = uv_mid * dx_safe

    cum_auc = np.concatenate([[0], np.cumsum(area_elements)])
else:
    cum_auc = np.array([0.0])

# Helper function moved outside loop
def find_min_in_range(arr, start, end):
    if start >= end:
        return start
    idx = np.argmin(arr[start:end])
    return start + idx

for run_id in unique_runs:
    # Ensure run_id is treated as float/int consistently
    try:
        run_id_val = float(run_id)
    except (ValueError, TypeError):
        continue

    start_idx = np.searchsorted(run_no_arr, run_id_val, side='left')
    end_idx = np.searchsorted(run_no_arr, run_id_val, side='right')

    if start_idx >= end_idx:
        continue

    # Extract run data
    run_vol = volume_arr[start_idx:end_idx]
    run_uv = uv_arr[start_idx:end_idx]

    # Step 4: Peak Detection
    max_uv = np.max(run_uv)
    prominence = max_uv * 0.1 if max_uv > 0 else 0.1

    peaks, _ = find_peaks(run_uv, prominence=prominence)

    if len(peaks) == 0:
        continue

    # Vectorized Segment Boundary Detection
    # Find all local minima in the run
    if len(run_uv) > 2:
        local_minima_indices = argrelextrema(run_uv, np.less)[0]
    else:
        local_minima_indices = np.array([])

    # Construct boundaries list

```



```

boundaries = []

# Refined segment boundaries for first and last peaks
# First segment: locate local minimum before first peak (or use fixed
# small window)
if len(local_minima_indices) > 0:
    # Find the first minimum that is before the first peak
    first_peak_idx = peaks[0]
    # Search for a minimum in [0, first_peak_idx]
    # If no minimum exists before the peak, use 0
    valid_mins_before = local_minima_indices[local_minima_indices <
        first_peak_idx]
    if len(valid_mins_before) > 0:
        boundaries.append(valid_mins_before[-1])
    else:
        boundaries.append(0)
else:
    boundaries.append(0)

# Boundaries between peaks
for i in range(len(peaks) - 1):
    p1, p2 = peaks[i], peaks[i+1]
    # Find min between p1 and p2
    min_idx = find_min_in_range(run_uv, p1, p2)
    boundaries.append(min_idx)

# Last segment: locate local minimum after last peak (or use fixed small
# window)
if len(peaks) > 0:
    last_peak_idx = peaks[-1]
    valid_mins_after = local_minima_indices[local_minima_indices >
        last_peak_idx]
    if len(valid_mins_after) > 0:
        boundaries.append(valid_mins_after[0])
    else:
        boundaries.append(len(run_uv))
else:
    boundaries.append(len(run_uv))

# Ensure boundaries are sorted and unique
boundaries = sorted(list(set(boundaries)))

max_segment_auc = -1
best_centroid_vol = None
valid_peak_found = False

for i in range(len(boundaries) - 1):
    seg_start = boundaries[i]
    seg_end = boundaries[i+1]

    if seg_end <= seg_start:
        continue

    # Check if this segment contains a peak
    # We assume the segment construction ensures this, but verify
    # A segment is valid if it contains at least one peak index
    segment_contains_peak = False
    for p in peaks:
        if seg_start <= p < seg_end:

```

```

        segment_contains_peak = True
        break

    if not segment_contains_peak:
        continue

    # Calculate AUC using pre-computed cumulative sums
    global_start = start_idx + seg_start
    global_end = start_idx + seg_end

    if global_start >= len(cum_auc) or global_end > len(cum_auc):
        continue

    seg_auc = cum_auc[global_end] - cum_auc[global_start]

    if seg_auc <= 0:
        continue

    # Calculate Centroid using Discrete Sum Formula
    # Slicing is required for discrete sum unless we pre-calc discrete
    # cumsums
    # We use slicing here for correctness as per feedback
    seg_vol_slice = run_vol[seg_start:seg_end]
    seg_uv_slice = run_uv[seg_start:seg_end]

    if len(seg_vol_slice) < 2:
        continue

    sum_uv = np.sum(seg_uv_slice)
    sum_vol_uv = np.sum(seg_vol_slice * seg_uv_slice)

    if sum_uv > 1e-9:
        centroid_vol = sum_vol_uv / sum_uv
    else:
        # Refined fallback: use midpoint or max UV volume
        # Ensure segment has at least two points (checked above)
        centroid_vol = seg_vol_slice[np.argmax(seg_uv_slice)]

    if seg_auc > max_segment_auc:
        max_segment_auc = seg_auc
        best_centroid_vol = centroid_vol
        valid_peak_found = True

    if valid_peak_found and max_segment_auc > AUC_THRESHOLD:
        retention_volumes.append(best_centroid_vol)
        run_ids.append(run_id)

# Step 5: Calculate Statistics
if len(retention_volumes) > 0:
    retention_volumes = np.array(retention_volumes)
    mean_vol = np.mean(retention_volumes)
    std_vol = np.std(retention_volumes, ddof=1)
    rsd_pct = (std_vol / mean_vol) * 100 if mean_vol != 0 else 0

    deviations = np.abs(retention_volumes - mean_vol) / mean_vol * 100
    flagged_mask = deviations > USP_THRESHOLD_PCT
    flagged_count = np.sum(flagged_mask)

    report_lines = [

```

```

        """
        f"-_**Total_Valid_Runs_Analyzed**:_{len(retention_volumes)}",
        f"-_**Mean_Retention_Volume**:_{mean_vol:.4f}_mL",
        f"-_**Calculated_RSD**:_{rsd_pct:.4f}%",
        f"-_**Flagged_Runs_(>{USP_THRESHOLD_PCT}%_deviation)**:_{flagged_count}",
        ""
    ]
    report_text = "\n".join(report_lines[:20])

    with open(REPORT_FILE, 'a') as f:
        f.write(report_text)

    plt.figure(figsize=(10, 6))
    plt.boxplot(retention_volumes, patch_artist=True, showmeans=True)

    plt.axhline(mean_vol, color='blue', linestyle='--', linewidth=2, label=f'Mean_({mean_vol:.2f}_mL)')

    upper_limit = mean_vol * (1 + USP_THRESHOLD_PCT/100)
    lower_limit = mean_vol * (1 - USP_THRESHOLD_PCT/100)
    plt.axhline(upper_limit, color='red', linestyle='--', linewidth=2, label=f'USP_Limit_({USP_THRESHOLD_PCT}%)')
    plt.axhline(lower_limit, color='red', linestyle='--', linewidth=2, label=f'USP_Limit_(-{USP_THRESHOLD_PCT}%)')

    plt.title('Retention_Volume_Variability_and_USP_Compliance')
    plt.ylabel('Retention_Volume_(mL)')
    plt.xlabel('Runs')
    plt.legend()
    plt.grid(axis='y', linestyle=':', alpha=0.6)

    plt.savefig(PLOT_FILE)
    plt.close()
else:
    report_lines = [
        """
        """
        f"-_**Total_Valid_Runs_Analyzed**:_0",
        f"-_**Mean_Retention_Volume**:_N/A",
        f"-_**Calculated_RSD**:_N/A",
        f"-_**Flagged_Runs**:_0",
        ""
    ]

    report_text = "\n".join(report_lines[:20])

    with open(REPORT_FILE, 'a') as f:
        f.write(report_text)
    print("No_valid_retention_volumes_found_to_plot.")

```

Listing 3: Code_3